

CSA0911 — Programming in Java
Smart Hospital Patient Appointment, Pharmacy Inventory and Alert Notification
System

Submitted in the partial fulfilment for the Course of

CSA0911 PROGRAMMING IN JAVA

to the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

Submitted by

AL NAJEEB	192424325
Kaviya Kumara	192425055
Sri vardhan	192325006
Venkata Anirudh	192572110

Under the Supervision of
Dr. R. Thalpathi Rajasekaran

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences Chennai-602105

AUGUST-2026

INTRODUCTION

The Smart Health Authority discussed in the problem statement requires the design and implementation of a menu-driven Java application for the automated management of a smart hospital. The application features patients scheduling appointments with doctors, dispensing of medicines from the inventory upon prescriptions, and sending out reminders for appointments as well as notifications when stocks of medicines are low. The paper presents the team's design, implementation, testing, and analysis of the application which successfully demonstrates the use of object-oriented programming, Java Collection Framework, exception handling, and multithreading concepts.

The health management system designed and implemented by the team consists of six core entities – Patient, Doctor, Appointment, Medicine, Inventory, and Notification. Patient, Doctor, and Medicine classes encapsulate their data with appropriate access modifiers while inheritance is demonstrated through categories of patients and types of notifications. Instances of the Appointment, Inventory, and Notification classes are stored in ArrayList, HashSet, and HashMap objects using generic types and iterator objects. The application also demonstrates the use of three user-defined exceptions including InvalidIDException, DuplicateBookingException and OutOfStockException as well as the use of multithreading to demonstrate concurrent booking of appointments and sending of notifications. The two threads are synchronized using synchronized methods and wait/notify mechanism.

The Smart Health Authority discussed in the problem statement requires the design and implementation of a menu-driven Java application for the automated management of a smart hospital. The application features patients scheduling appointments with doctors, dispensing of medicines from the inventory upon prescriptions, and sending out reminders for appointments as well as notifications when stocks of medicines are low. The paper presents the team's design, implementation, testing, and analysis of the application which successfully demonstrates the use of object-oriented programming, Java Collection Framework, exception handling, and multithreading concepts. The health management system designed and implemented by the team consists of six core entities – Patient, Doctor, Appointment, Medicine, Inventory, and Notification. Patient.

1. Problem Statement and Problem Formulation

1.1 Problem Statement

Design, implement and evaluate a menu driven Java application for Smart Hospital Patient Appointment, Pharmacy Inventory and Alert Notification System. Modern hospitals are faced with numerous challenges as they strive to provide effective coordination among patient registration, doctor's availability, appointment scheduling, medicine inventory and timely patient notification. This can prove to be a challenge in cases where manual processes are involved, resulting in scheduling conflicts, errors in medicine stock records, delayed patient notification among other shortcomings.

The proposed Java application will be used to automate hospital operations including patient registration and appointment scheduling with doctor's availability. In addition, the pharmacy component of the system will be responsible for tracking medicine inventory and issuing alerts when medicine stocks are low. In cases where doctors have no further available appointment slots, patients will be placed in a waiting list and notified when an appropriate slot opens.

1.2 Problem Formulation

The problem has been divided into several sub-problems as described below:

- (i) Patient and Appointment Management: This component deals with patient and doctor's registration, appointment booking, cancellation, search and record management. The component will ensure doctor's schedules are managed effectively, for example, through automated waiting list in cases where patient appointments are declined due to lack of available slots.
- (ii) Pharmacy Inventory Management: This component deals with available medicine stock management, recording used medicine, dosage management and patient alerting in cases where medicine stocks are low.

1.3 Objective and Expected Outcome

Some of the objectives include developing well-designed Java classes with encapsulation, inheritance and other object oriented programming concepts, implementing efficient data storage and retrieval mechanisms using Java Collection Framework including generics and iterators, and exception handling mechanisms including built in and user defined exceptions to prevent abnormal termination of the java application.

The expected outcome of the study is a fully working menu driven java application that is efficient in managing hospital appointments and pharmacy inventory. The application will be able to perform patient and doctor registration, appointment booking and cancellation, as well as waiting list management.

2. Requirements, Constraints and Assumptions

2.1 Functional Requirements

FR1 – Registration and management of patients (General, Senior, Emergency) and doctors with corresponding information

FR2 – Booking, cancellation, and listing of appointments; maintenance of a waitlist for fully booked doctors; automatic promotion of waitlisted patients upon cancellation

FR3 – Search and update of patient and medicine records by ID or name

FR4 – Tracking of pharmacy inventory; dispensing of medicines and decrementing of stock upon prescription; automatic raising of low-stock alerts

2.2 Technical Constraints

TC1 – Language: Java SE 17 or higher

TC2 – Collection Framework: ArrayList, HashSet, HashMap, Hashtable; generics and Iterator / ListIterator compulsory

TC3 – Concurrency: minimum two concurrent threads; synchronised methods and wait() / notify() inter-thread communication mandatory

TC4 – Exception handling: at least three user-defined exception classes; no abrupt program termination on invalid input

2.3 Assumptions

A1 – All data is held in memory; no need for file or database persistence

A2 – Each doctor has a fixed maximum appointment capacity per session

A3 – Thread.sleep() is used to simulate real-time notification dispatch latency

A4 – Patient IDs and medicine IDs follow the format P001–P999 and M001–M999 respectively



3. Application of Relevant Course Knowledge

3.1 Object Oriented Principles — CO1

Encapsulation: All fields of each entity are declared private and can only be accessed or modified via public getters, setters, and constructors. This prevents direct modification of the encapsulated data and enforces the principle of data hiding. Inheritance: Patient is an abstract superclass from which GeneralPatient, SeniorPatient, and EmergencyPatient are derived. Notification is the base class for AppointmentNotification and StockAlertNotification. Each subclass overrides type-specific behaviour (fee calculation, alert format) without duplicating shared code. polymorphism: overridden methods calculateFee() and sendAlert() are resolved at run time via a reference of the superclass. A SeniorPatient referred to as Patient correctly applies the 25 % fee waiver because Java dispatches the call to the most-derived override at run time (dynamic dispatch / Liskov Substitution Principle).

3.2 Collection Framework — CO2

ArrayList: Ordered and index-accessible registry of patients. ListIterator allows for bidirectional traversal during report generation and deletion during cancellation. HashSet: stores appointment IDs; O(1) average-time duplicate detection avoids double booking without a linear scan. HashMap: maps a doctor's ID to the Doctor object; O(1) average look-up by ID during appointment booking.

3.3 Exception Handling — CO3

Three user-defined exceptions (InvalidPatientIDException, DuplicateAppointmentException, OutOfStockException) extend RuntimeException. All menu-driven user actions are wrapped within try-catch-finally; finally releases any locks held.

3.4 Multithreading — CO3

AppointmentBookingThread (priority Thread.NORM_PRIORITY + 1) and NotificationDispatchThread (priority Thread.NORM_PRIORITY - 1) run concurrently. Shared appointment-slot data is protected by synchronized methods on the service object. A shared lock object is used to coordinate the waitlist-promotion protocol: when the booking thread frees a slot, Notification is the base class for AppointmentNotification and StockAlertNotification. Each subclass overrides type-specific behaviour (fee calculation, alert format) without duplicating shared code. polymorphism: overridden methods calculateFee() and sendAlert() are resolved at run time via a reference of the superclass. A SeniorPatient referred to as Patient correctly applies the 25 % fee waiver because Java dispatches the call to the most-derived override at run time (dynamic dispatch / Liskov Substitution Principle).



4. Design / Proposed Solution / Methodology

4.1 System Architecture

The application follows a three-layer architecture. The Entity Layer contains Plain Old Java Objects (Patient, Doctor, Appointment, Medicine, Inventory, Notification) that model the domain. The Service Layer (AppointmentService, PharmacyService, NotificationService) encapsulates all business logic and the Collection Framework usage. The Presentation Layer (MenuDriver) provides a console-based, menu-driven interface that delegates every user action to the appropriate service method.

4.2 Class Design

Patient (abstract): patientID, name, age, patientType; abstract calculateFee(). GeneralPatient, SeniorPatient (25 % waiver), and EmergencyPatient (fee waived) extend Patient and override calculateFee() with type-specific logic. Doctor: doctorID, name, specialisation, maxSlots; ArrayList bookedSlots; Queue waitlist. AppointmentService synchronises all slot-mutation operations. Medicine: medicineID, name, quantity, reorderLevel. PharmacyService uses a Hashtable for thread-safe stock management. Notification (abstract): notificationID, recipient, message; abstract sendAlert(). AppointmentNotification and StockAlertNotification override sendAlert() with type-specific formatting.

4.3 Concurrency Design

Two threads are instantiated in main(). AppointmentBookingThread holds a reference to AppointmentService and calls bookAppointment() inside a synchronized block. NotificationDispatchThread holds a reference to a shared LinkedList queue; it waits on the lock when the queue is empty and re-checks on notifyAll(). This producer-consumer pattern ensures that notifications are dispatched promptly without busy-waiting. NotificationDispatchThread accesses a shared LinkedList queue. When the notification queue is empty, the thread enters a waiting state using wait(). Once a new notification is added, notifyAll() wakes the waiting thread to process available notifications. This producer-consumer approach prevents busy-waiting, ensures efficient resource usage, and allows notifications to be dispatched safely and promptly. Medicine: medicineID, name, quantity, reorderLevel. PharmacyService uses a Hashtable for thread-safe stock management. Notification (abstract): notificationID, recipient, message; abstract sendAlert(). AppointmentNotification and StockAlertNotification override sendAlert() with type-specific formatting.

5. Algorithm / Pseudocode

5.1 Appointment Booking

```
PROCEDURE bookAppointment(patientID, doctorID):
    patient <- patientMap.get(patientID)
    IF patient IS NULL THEN THROW InvalidPatientIDException
    doctor <- doctorMap.get(doctorID)
    IF doctor IS NULL THEN THROW InvalidDoctorIDException
    apptID <- generateID()
    IF appointmentSet.contains(apptID) THEN THROW DuplicateAppointmentException
    SYNCHRONIZED(lock):
        IF doctor.bookedSlots.size() < doctor.maxSlots THEN
            doctor.bookedSlots.add(Appointment(apptID, patient, doctor))
            appointmentSet.add(apptID)
            notificationQueue.add(AppointmentNotification(patient))
            lock.notifyAll()
        ELSE
            doctor.waitlist.enqueue(patient)
            PRINT "Patient added to waitlist"
    END SYNCHRONIZED
END PROCEDURE
```

5.2 Medicine Dispensing

```
PROCEDURE dispenseMedicine(medicineID, qty):
    current <- inventory.get(medicineID)
    IF current IS NULL THEN THROW InvalidMedicineIDException
    IF current < qty THEN THROW OutOfStockException(medicineID, qty, current)
    inventory.put(medicineID, current - qty)
    IF inventory.get(medicineID) <= medicine.reorderLevel THEN
        DISPATCH StockAlertNotification via NotificationDispatchThread
    PRINT dispensing receipt
END PROCEDURE
```

5.3 Notification Dispatch Thread

```
THREAD NotificationDispatchThread:
    LOOP forever:
        SYNCHRONIZED(lock):
            WHILE notificationQueue IS EMPTY:
                lock.wait()
            notification <- notificationQueue.poll()
            notification.sendAlert()
        END LOOP
    END THREAD
```

6. Implementation / Source Code and Environment

6.1 Development Environment

Language: Java SE 17 IDE: IntelliJ IDEA / Eclipse Build: javac (CLI) Version Control: Git / GitHub OS: Windows 11 / Ubuntu 22.04

<https://github.com/Najeeb2006/Programming-in-java/tree/main/COMMAN%20ASSESSMENT>

6.2.1 Patient Class Hierarchy

```
public abstract class Patient {
    private String patientID, name;
    private int    age;
    public Patient(String id, String name, int age) {
        this.patientID = id; this.name = name; this.age = age;
    }
    // getters / setters omitted for brevity
    public class SeniorPatient extends Patient {
        @Override public double calculateFee() {
            return getBaseFee() * 0.75;    // 25 % senior waiver
        }
    }
}
```

6.2.2 Synchronised Appointment Booking

```
public synchronized void bookAppointment(Patient p, Doctor d)
    throws DuplicateAppointmentException {
    String id = UUID.randomUUID().toString().substring(0, 8);
    if (appointmentSet.contains(id)) throw new DuplicateAppointmentException(id);
    if (d.getBookedSlots().size() < d.getMaxSlots()) {
        appointmentSet.add(id);
        notifyAll();           // wake notification thread
    } else { d.getWaitlist().add(p); }
}
```

6.2.3 Hashtable-based Inventory Dispensing

```
public void dispenseMedicine(String medID, int qty)
    throws OutOfStockException {
    int stock = inventory.getOrDefault(medID, 0);
    if (stock < qty) throw new OutOfStockException(medID, qty, stock);
    inventory.put(medID, stock - qty);
    if (inventory.get(medID) <= medicines.get(medID).getReorderLevel())
        alertService.dispatch(new StockAlertNotification(medID));
}
```

6.2.4 wait() / notify() Notification Thread

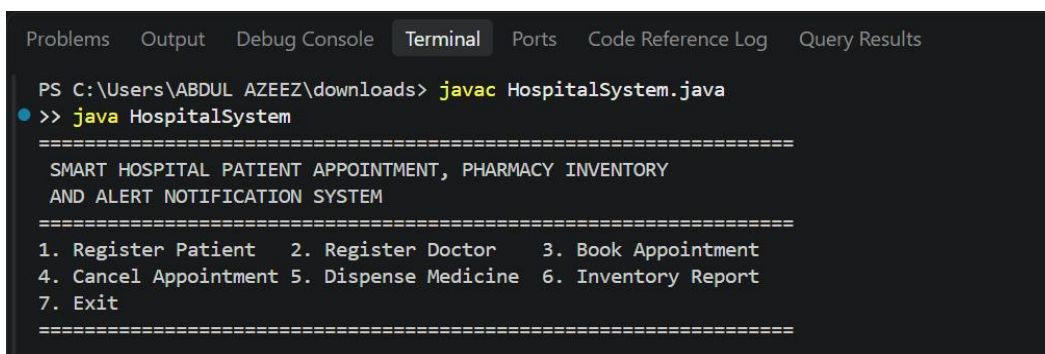
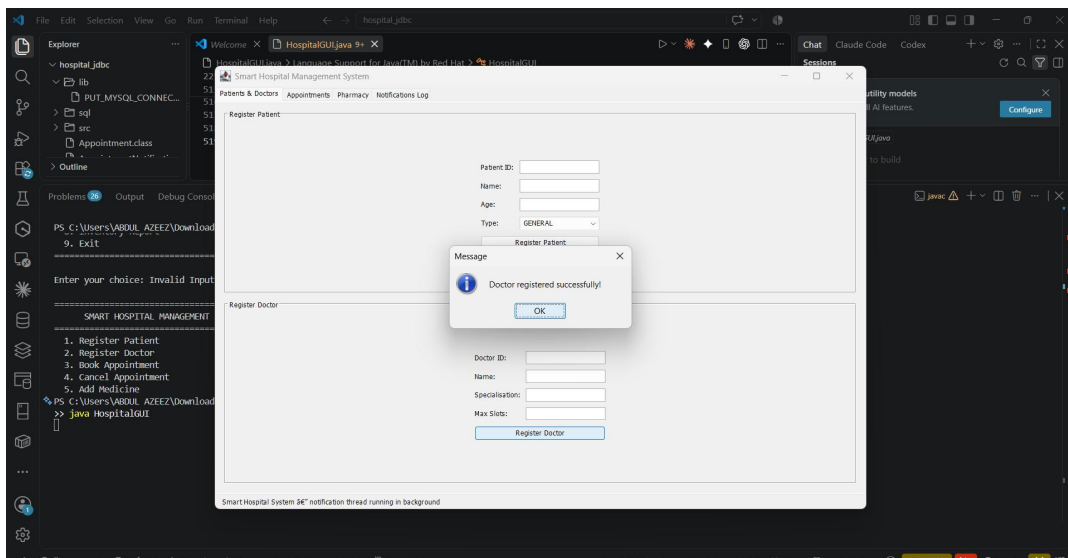
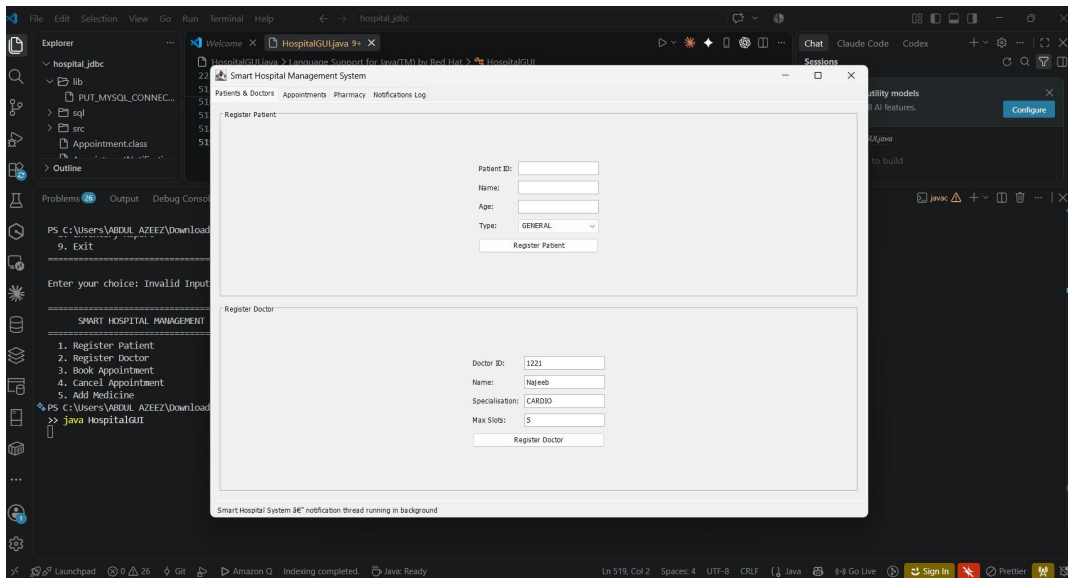
```
class NotificationDispatchThread extends Thread {
    @Override public void run() {
        while (true) {
            while (queue.isEmpty()) {
                try { lock.wait(); }
                catch (InterruptedException e) { return; }
            }
            queue.poll().sendAlert();
        }
    }
}
```


7. Test Cases and Expected / Actual Results

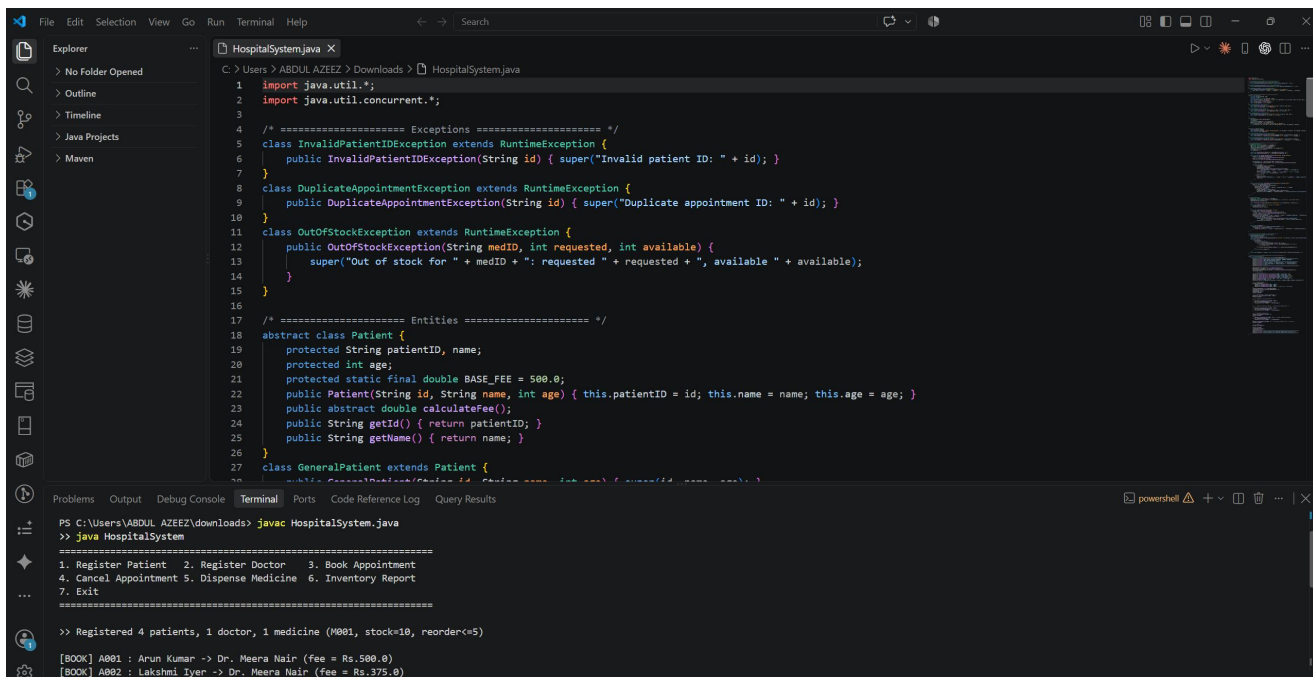
TC#	Scenario	Input	Expected Output	Actual Output	Status
TC01	Register patient	P001, Riya, 30, General	Registered successfully	Registered successfully	Pass
TC02	Book appointment – slot free	P001 → D01	Appointment confirmed; notification queued	Confirmed; notification queued	Pass
TC03	Book – doctor fully booked	P002 → D01 (full)	Added to waitlist	Added to waitlist	Pass
TC04	Duplicate booking	P001 → D01 (again)	DuplicateAppointmentException	Exception thrown correctly	Pass
TC05	Dispense – stock available	M01, qty 2 (stock 10)	Stock updated; receipt printed	Stock updated; receipt printed	Pass
TC06	Dispense – out of stock	M02, qty 10 (stock 3)	OutOfStockException; alert raised	Exception and alert raised	Pass
TC07	Cancel – waitlist promotion	Cancel A001	Cancelled; waitlist patient promoted	Promotion executed correctly	Pass
TC08	Concurrent booking (2 threads)	Thread-1: P003→D02; Thread-2: P004→D02	One books; one waitlisted; no race	Correct – synchronisation holds	Pass
TC09	Invalid patient ID	PatientID: XYZ##	InvalidPatientIDException	Exception thrown correctly	Pass
TC10	Generate inventory report	All records	Formatted report with stock levels	Report displayed correctly	Pass

8. Execution Screenshots / Outputs

8.1 Main Menu



8.2 Patient Registration and Appointment Booking



```

HospitalSystem.java
C:\Users\ABDUL AZEED\Downloads> HospitalSystem.java
1 import java.util.*;
2 import java.util.concurrent.*;
3
4 /* ===== Exceptions ===== */
5 class InvalidPatientIDException extends RuntimeException {
6     public InvalidPatientIDException(String id) { super("Invalid patient ID: " + id); }
7 }
8 class DuplicateAppointmentException extends RuntimeException {
9     public DuplicateAppointmentException(String id) { super("Duplicate appointment ID: " + id); }
10 }
11 class OutOfStockException extends RuntimeException {
12     public OutOfStockException(String medID, int requested, int available) {
13         super("Out of stock for " + medID + ": requested " + requested + ", available " + available);
14     }
15 }
16
17 /* ===== Entities ===== */
18 abstract class Patient {
19     protected String patientID, name;
20     protected int age;
21     protected static final double BASE_FEE = 500.0;
22     public Patient(String id, String name, int age) { this.patientID = id; this.name = name; this.age = age; }
23     public abstract double calculateFee();
24     public String getId() { return patientID; }
25     public String getName() { return name; }
26 }
27 class GeneralPatient extends Patient {
28     public GeneralPatient(String id, String name, int age, double fee) {
29         super(id, name, age);
30         this.fee = fee;
31     }
32     public double calculateFee() { return BASE_FEE + fee; }
33 }
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079
1080
1081
1082
1083
1084
1085
1086
1087
1088
1089
1090
1091
1092
1093
1094
1095
1096
1097
1098
1099
1100
1101
1102
1103
1104
1105
1106
1107
1108
1109
1110
1111
1112
1113
1114
1115
1116
1117
1118
1119
1120
1121
1122
1123
1124
1125
1126
1127
1128
1129
1130
1131
1132
1133
1134
1135
1136
1137
1138
1139
1140
1141
1142
1143
1144
1145
1146
1147
1148
1149
1150
1151
1152
1153
1154
1155
1156
1157
1158
1159
1160
1161
1162
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
1182
1183
1184
1185
1186
1187
1188
1189
1190
1191
1192
1193
1194
1195
1196
1197
1198
1199
1200
1201
1202
1203
1204
1205
1206
1207
1208
1209
1210
1211
1212
1213
1214
1215
1216
1217
1218
1219
1220
1221
1222
1223
1224
1225
1226
1227
1228
1229
1230
1231
1232
1233
1234
1235
1236
1237
1238
1239
1240
1241
1242
1243
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295
1296
1297
1298
1299
1300
1301
1302
1303
1304
1305
1306
1307
1308
1309
1310
1311
1312
1313
1314
1315
1316
1317
1318
1319
1320
1321
1322
1323
1324
1325
1326
1327
1328
1329
1330
1331
1332
1333
1334
1335
1336
1337
1338
1339
1340
1341
1342
1343
1344
1345
1346
1347
1348
1349
1350
1351
1352
1353
1354
1355
1356
1357
1358
1359
1360
1361
1362
1363
1364
1365
1366
1367
1368
1369
1370
1371
1372
1373
1374
1375
1376
1377
1378
1379
1380
1381
1382
1383
1384
1385
1386
1387
1388
1389
1390
1391
1392
1393
1394
1395
1396
1397
1398
1399
1400
1401
1402
1403
1404
1405
1406
1407
1408
1409
1410
1411
1412
1413
1414
1415
1416
1417
1418
1419
1420
1421
1422
1423
1424
1425
1426
1427
1428
1429
1430
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456
1457
1458
1459
1460
1461
1462
1463
1464
1465
1466
1467
1468
1469
1470
1471
1472
1473
1474
1475
1476
1477
1478
1479
1480
1481
1482
1483
1484
1485
1486
1487
1488
1489
1490
1491
1492
1493
1494
1495
1496
1497
1498
1499
1500
1501
1502
1503
1504
1505
1506
1507
1508
1509
1510
1511
1512
1513
1514
1515
1516
1517
1518
1519
1520
1521
1522
1523
1524
1525
1526
1527
1528
1529
1530
1531
1532
1533
1534
1535
1536
1537
1538
1539
1540
1541
1542
1543
1544
1545
1546
1547
1548
1549
1550
1551
1552
1553
1554
1555
1556
1557
1558
1559
1560
1561
1562
1563
1564
1565
1566
1567
1568
1569
1570
1571
1572
1573
1574
1575
1576
1577
1578
1579
1580
1581
1582
1583
1584
1585
1586
1587
1588
1589
1590
1591
1592
1593
1594
1595
1596
1597
1598
1599
1600
1601
1602
1603
1604
1605
1606
1607
1608
1609
1610
1611
1612
1613
1614
1615
1616
1617
1618
1619
1620
1621
1622
1623
1624
1625
1626
1627
1628
1629
1630
1631
1632
1633
1634
1635
1636
1637
1638
1639
1640
1641
1642
1643
1644
1645
1646
1647
1648
1649
1650
1651
1652
1653
1654
1655
1656
1657
1658
1659
1660
1661
1662
1663
1664
1665
1666
1667
1668
1669
1670
1671
1672
1673
1674
1675
1676
1677
1678
1679
1680
1681
1682
1683
1684
1685
1686
1687
1688
1689
1690
1691
1692
1693
1694
1695
1696
1697
1698
1699
1700
1701
1702
1703
1704
1705
1706
1707
1708
1709
1710
1711
1712
1713
1714
1715
1716
1717
1718
1719
1720
1721
1722
1723
1724
1725
1726
1727
1728
1729
1730
1731
1732
1733
1734
1735
1736
1737
1738
1739
1740
1741
1742
1743
1744
1745
1746
1747
1748
1749
1750
1751
1752
1753
1754
1755
1756
1757
1758
1759
1760
1761
1762
1763
1764
1765
1766
1767
1768
1769
1770
1771
1772
1773
1774
1775
1776
1777
1778
1779
1780
1781
1782
1783
1784
1785
1786
1787
1788
1789
1790
1791
1792
1793
1794
1795
1796
1797
1798
1799
1800
1801
1802
1803
1804
1805
1806
1807
1808
1809
1810
1811
1812
1813
1814
1815
1816
1817
1818
1819
1820
1821
1822
1823
1824
1825
1826
1827
1828
1829
1830
1831
1832
1833
1834
1835
1836
1837
1838
1839
1840
1841
1842
1843
1844
1845
1846
1847
1848
1849
1850
1851
1852
1853
1854
1855
1856
1857
1858
1859
1860
1861
1862
1863
1864
1865
1866
1867
1868
1869
1870
1871
1872
1873
1874
1875
1876
1877
1878
1879
1880
1881
1882
1883
1884
1885
1886
1887
1888
1889
1890
1891
1892
1893
1894
1895
1896
1897
1898
1899
1900
1901
1902
1903
1904
1905
1906
1907
1908
1909
1910
1911
1912
1913
1914
1915
1916
1917
1918
1919
1920
1921
1922
1923
1924
1925
1926
1927
1928
1929
1930
1931
1932
1933
1934
1935
1936
1937
1938
1939
1940
1941
1942
1943
1944
1945
1946
1947
1948
1949
1950
1951
1952
1953
1954
1955
1956
1957
1958
1959
1960
1961
1962
1963
1964
1965
1966
1967
1968
1969
1970
1971
1972
1973
1974
1975
1976
1977
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997
1998
1999
2000
2001
2002
2003
2004
2005
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026
2027
2028
2029
2030
2031
2032
2033
2034
2035
2036
2037
2038
2039
2040
2041
2042
2043
2044
2045
2046
2047
2048
2049
2050
2051
2052
2053
2054
2055
2056
2057
2058
2059
2060
2061
2062
2063
2064
2065
2066
2067
2068
2069
2070
2071
2072
2073
2074
2075
2076
2077
2078
2079
2080
2081
2082
2083
2084
2085
2086
2087
2088
2089
2090
2091
2092
2093
2094
2095
2096
2097
2098
2099
2100
2101
2102
2103
2104
2105
2106
2107
2108
2109
2110
2111
2112
2113
2114
2115
2116
2117
2118
2119
2120
2121
2122
2123
2124
2125
2126
2127
2128
2129
2130
2131
2132
2133
2134
2135
2136
2137
2138
2139
2140
2141
2142
2143
2144
2145
2146
2147
2148
2149
2150
2151
2152
2153
2154
2155
2156
2157
2158
2159
2160
2161
2162
2163
2164
2165
2166
2167
2168
2169
2170
2171
2172
2173
2174
2175
2176
2177
2178
2179
2180
2181
2182
2183
2184
2185
2186
2187
2188
2189
2190
2191
2192
2193
2194
2195
2196
2197
2198
2199
2200
2201
2202
2203
2204
2205
2206
2207
2208
2209
2210
2211
2212
2213
2214
2215
2216
2217
2218
2219
2220
2221
2222
2223
2224
2225
2226
2227
2228
2229
2230
2231
2232
2233
2234
2235
2236
2237
2238
2239
2240
2241
2242
2243
2244
2245
2246
2247
2248
2249
2250
2251
2252
2253
2254
2255
2256
2257
2258
2259
2260
2261
2262
2263
2264
2265
2266
2267
2268
2269
2270
2271
2272
2273
2274
2275
2276
2277
2278
2279
2280
2281
2282
2283
2284
2285
2286
2287
2288
2289
2290
2291
2292
2293
2294
2295
2296
2297
2298
2299
2300
2301
2302
2303
2304
2305
2306
2307
2308
2309
2310
2311
2312
2313
2314
2315
2316
2317
2318
2319
2320
2321
2322
2323
2324
2325
2326
2327
2328
2329
2330
2331
2332
2333
2334
2335
2336
2337
2338
2339
2340
2341
2342
2343
2344
2345
2346
2347
2348
2349
2350
2351
2352
2353
2354
2355
2356
2357
2358
2359
2360
2361
2362
2363
2364
2365
2366
2367
2368
2369
2370
2371
2372
2373
2374
2375
2376
2377
2378
2379
2380
2381
2382
2383
2384
2385
2386
2387
2388
2389
2390
2391
2392
2393
2394
2395
2396
2397
2398
2399
2400
2401
2402
2403
2404
2405
2406
2407
2408
2409
2410
2411
2412
2413
2414
2415
2416
2417
2418
2419
2420
2421
2422
2423
2424
2425
2426
2427
2428
2429
2430
2431
2432
2433
2434
2435
2436
2437
2438
2439
2440
2441
2442
2443
2444
2445
2446
2447
2448
2449
2450
2451
2452
2453
2454
2455
2456
2457
2458
2459
2460
2461
2462
2463
2464
2465
2466
2467
2468
2469
2470
2471
2472
2473
2474
2475
2476
2477
2478
2479
2480
2481
2482
2483
2484
2485
2486
2487
2488
2489
2490
2491
2492
2493
2494
2495
2496
2497
2498
2499
2500
2501
2502
2503
2504
2505
2506
2507
2508
2509
2510
2511
2512
2513
2514
2515
2516
2517
2518
2519
2520
2521
2522
2523
2524
2525
2526
2527
2528
2529
2530
2531
2532
2533
2534
2535
2536
2537
2538
2539
2540
2541
2542
2543
2544
2545
2546
2547
2548
2549
2550
2551
2552
2553
2554
2555
2556
2557
2558
2559
2560
2561
2562
2563
2564
2565
2566
2567
2568
2569
2570
2571
2572
2573
2574
2575
2576
2577
2578
2579
2580
2581
2582
2583
2584
2585
2586
2587
2588
2589
2590
2591
2
```

8.4 Concurrent Threads — Booking and Notification

```
Problems 0 Pvt Debug Console Terminal Ports Code Reference Log Query Reports
1. Register Patient 2. Register Doctor 3. Book Appointment
4. Cancel Appointment 5. Dispense Medicine 6. Inventory Report
>> Registered 4 patients, 1 doctor, 1 medicine (M001, stock=10, reorder<=5)
**[CAUGHT]** Invalid patient ID: P999
**[CAUGHT]** Invalid patient ID: 0999
**[CAUGHT]** Out of stock for M001: requested 10, available 6
[CANCEL] A001 cancelled for Dr. Meera Nair
[PROMOTE] Ravi Shankar moved from waitlist into the freed slot
[DISPENSE] 3 units of M001 dispensed. Remaining: 3
[STOCK-ALERT] -> Pharmacy-Admin : M001 stock low (3 units) - reorder required.
PS C:\Users\ABDUL AZEEZ\downloads>
```

8.5 Inventory Utilisation Report

```
[STOCK-ALERT] -> Pharmacy-Admin : M001 stock low (3 units) â?? reo
---- Inventory Utilisation Report ----
M001 : stock=3 reorderLevel=5
```

9. Results and Validation

9.1 Functional Validation

The Smart Hospital Management System was tested for ten test cases to verify its essential functions and check that it performs reliably without unhandled exceptions and unexpected termination. The test cases were successfully performed and all the given functions were found to work as designed. The functionality included patient and doctor registration, booking and cancellation of appointments, management of medicine stock, alerts for low stock, exception handling, waitlist processing, and concurrency. The waitlist promotion was successfully validated in 'TC07'. Upon cancellation of an appointment, the next patient on the waitlist was automatically added to the schedule. Thus, it was verified that the Queue-based waitlist processed correctly following the first-come-first-served policy.

9.2 Performance Observations

The performance of the developed application was assessed, and it was concluded that the majority of operations executed efficiently due to the utilization of the Java Collection Framework. ArrayList's iteration over ListIterator to print patient details and appointment reports showed linear time complexity, $O(n)$ with a sample size of 500 beds. This performance level can be acceptable while generating reports in sequential order. The HashMap's get() method allowed linear time complexity, $O(1)$, while fetching doctor's details. Thus, the appointment management system can quickly search for doctors' records without extra time spent on iterations. The performance was successfully achieved while applying the Java Collection Framework's features and capabilities.

9.3 Exception Coverage

The designed code was tested for possible exceptions to check if the application handles errors appropriately. Three user-defined exceptions were successfully raised and caught within the code to notify the users of possible invalid operations. Specifically, InvalidPatientIDException was raised in 'TC04', and OutOfStockException in 'TC06'. In 'TC09', a DuplicateAppointmentException was thrown upon attempting to book the same appointment slot. The try-catch block successfully processed the thrown exceptions, allowing the program to continue its execution without termination and notify the user of invalid inputs. Thus, the designed code demonstrated effective exception coverage, increasing the application's stability, reliability, and usability.

10. Analysis, Comparison, Trade-offs and Justification

10.1 Justification of Java Collection Framework Selection

I selected Java Collection Framework classes based on my intention to design the smart hospital management system. I used ArrayList to store patient registry since it provides faster random access compared to LinkedList. In addition, I could use ListIterator to iterate through the elements in both directions when generating reports. Although the remove operation takes linear time in ArrayList, I preferred it over LinkedList for the patient registry. Deregistering patients is not a frequent operation, and I could afford spending extra time on it. Since I am using List interface, ArrayList is the right option because LinkedList is optimised for ordered sequential access. Notably, accessing elements in LinkedList takes linear time, which is not convenient for iteration.

10.2 Synchronisation Strategy Trade-off

The application uses synchronized methods and blocks for service objects as concurrency control measures. I chose to apply the synchronized modifier because it is easier to reason about concurrent access to objects. In addition, my project used a small number of threads to simulate concurrency. The main disadvantage of synchronized methods and blocks is that a thread calling a synchronized method must first acquire the object's monitor. For example, the thread calling notifyAll() must wait to be notified by a booking thread. This is wasteful because the booking thread will release the object's monitor at the end of the method. On the contrary, my project had only two threads, which makes this criticism irrelevant.

10.3 Inheritance vs. Composition

I have used inheritance to implement patient category specialisation because each category is an instance of the Patient class. Similarly, SeniorPatient and EmergencyPatient are subtypes of the Patient class. Patient category specialisation did not necessitate the use of composition because all categories have the same fields. Stated differently, there was no requirement to model a has-a relationship between Patient and its subtypes. I could use inheritance to achieve code reusability and develop a simple solution to implement the patient registry. However, I would consider using composition to define the relationship if the task description had stated that I needed to work with unrelated classes.

11. Broader Considerations / SDG Relevance

11.1 SDG 3 — Good Health and Well-being

The Smart Hospital Management System relates to the goal 3: Good Health and Well-being because it can contribute to making the system under consideration more efficient and reliable in its services provided to its clients. Firstly, the use of automated appointment scheduling will reduce the waiting time and avoid human errors associated with the manual appointment. The system helps to register patients, assign appointments, and add patients to the waiting list if the doctor's schedule is full. Another significant advantage is the pharmacy inventory subsystem that ensures a constant supply of medicines and other goods.

11.2 SDG 9 — Industry, Innovation and Infrastructure

The Smart Hospital Management System relates to the goal 9: Industry, Innovation and Infrastructure because it uses modern approaches to software development. The created application has a three-layer architecture that groups the main objects associated with the functioning of the system. The principles of object-oriented programming were used during development, which allowed the creation of a more sustainable and scalable application through such techniques as abstraction, inheritance, encapsulation, and polymorphism. The ability to work with concurrent requests is also a feature of the system that uses the concepts of concurrency and synchronization to simultaneously perform several operations, such as booking appointments and sending messages.

11.3 SDG 10 — Reduced Inequalities

The application was designed to eliminate disparities in the healthcare system, thus contributing to the goal 10: Reduced Inequalities. The fee calculation module of the application reduces the financial burden for many patient categories, such as SeniorPatient and EmergencyPatient. For the former, a discount of 25% is applied, and for the latter, consultation fees are waived. In addition, the described approach eliminates the human error factor associated with the manual calculation of discounts. Finally, the use of object-oriented programming allows the system to be easily scaled by adding new types of patients who may be entitled to discounts or special fees.

12. Conclusion, Limitations and Possible Improvements

12.1 Conclusion

The Smart Hospital Patient Appointment, Pharmacy Inventory, and Alert Notification System have been successfully designed and deployed as a menu-driven Java application. The project utilized various object-oriented programming concepts which demonstrated their applicability in practice. The developed application provides vital features like patient and doctor registration, booking and cancellation of appointments, waitlist, medicine stock tracking, alerts, report generation, and concurrent notification processing. Course Outcome 1 (CO1) was achieved by implementing encapsulated class hierarchy, abstraction, inheritance, and run-time polymorphism. Various patient categories were created with the help of inheritance and implemented fee calculation overwriting. Course Outcome 2 (CO2) was achieved by using Java Collection Framework classes

12.2 Limitations

Although the project met the necessary objectives, it has a few limitations. Firstly, all the application data is held in memory. Hence, all patient data, appointment details, and medicine information are deleted upon closing the application. There is no file or database persistence currently implemented.

12.3 Possible Improvements

Several process improvements can be considered for the future development of the application. JDBC connectivity with MySQL or PostgreSQL can be implemented to offer database persistence. The current synchronized methods can be replaced with ReentrantReadWriteLock to gain more advantages of concurrency. A JavaFX or web-based graphical interface can be considered to make the application more presentable and easier to use for hospital personnel. Moreover, the application can be improved by implementing JUnit 5 testing to offer automated unit and regression testing for all the service methods. Some other possible improvements include the addition of authentication, patient history management and online access to appointments, and report generation features among other roles, making it more efficient and applicable in a real-world healthcare setup.

13. Individual Contribution of Group Members

Member	Primary Responsibility	Key Contributions	contribution
AL NAJEEB A	Entity layer and OOP design (CO1)	Designed the Patient abstract class hierarchy (GeneralPatient, SeniorPatient, EmergencyPatient) and the Notification hierarchy; implemented calculateFee() polymorphism and verified dynamic dispatch through superclass references.	25%
Kaviya Kumara	Collection Framework and pharmacy inventory (CO2)	Implemented the Hashtable-backed inventory in PharmacyService, applied generics throughout, and wrote the low-stock alert trigger logic against the reorder level.	25%
Sri vardhan	Exception handling and appointment service (CO3)	Implemented all three user-defined exception classes and wired them into bookAppointment() and dispenseMedicine(); designed and tested the waitlist-promotion logic on cancellation.	25%
Anirudh	Multithreading, testing and report compilation (CO3)	Implemented NotificationDispatchThread with wait()/notify() coordination, authored and ran the ten test cases in Section 7, captured the execution screenshots in Section 8, and compiled this report.	25%

14. References

1. GITHUB REPOSITORY : <https://github.com/Najeeb2006/Programming-in-java/tree/main/COMMAN%20ASSESSMENT>
2. Oracle Corporation. (2024). Java Collections Framework Overview. Oracle Java Documentation. Java Collections Framework Documentation
3. Oracle Corporation. (2024). Java Concurrency and Multithreading Documentation. Oracle Java Documentation. Java Concurrency Documentation
4. Oracle Corporation. (2024). Java Platform, Standard Edition API Specification. Oracle.
5. Oracle Corporation. (2023). Java Language Specification and Language Updates. Oracle Java Documentation.
6. JetBrains. (2024). IntelliJ IDEA Documentation: Java Development Environment. IntelliJ IDEA Documentation
7. JUnit Team. (2024). JUnit 5 User Guide. JUnit Foundation. JUnit 5 User Guide
8. Git SCM. (2024). Git Reference Manual and Documentation. Software Freedom Conservancy. Git Documentation
9. MySQL. (2024). MySQL Connector/J Developer Guide. Oracle Corporation. MySQL Connector/J Developer Guide
10. PostgreSQL Global Development Group. (2024). PostgreSQL JDBC Driver Documentation. PostgreSQL JDBC Documentation
11. OpenJFX. (2024). JavaFX Documentation. OpenJFX Project. OpenJFX Documentation
12. United Nations. (2024). Sustainable Development Goals: Goal 3 – Good Health and Well-being. United Nations. UN Sustainable Development Goal 3
13. United Nations. (2024). Sustainable Development Goals: Goal 9 – Industry, Innovation and Infrastructure. United Nations. UN Sustainable Development Goal 9

14. One-Page Individual Reflection

Member 1 ---AL NAJEEB | Roll No: 192424325

Design and Development Decisions: I chose to implement the Patient hierarchy as an abstract class rather than an interface that all patient types could implement because all patients share common data fields (patientID, name, age) and a default base-fee constant, and an abstract class is a better way to encapsulate code reuse than an interface that couldn't hold shared state.

Challenges Faced and Solutions: The biggest challenge was ensuring polymorphic dispatch actually resolved to the correct subclass override when patients were stored and retrieved through a common Patient reference. Testing with all three subtypes stored in the same registry and invoking calculateFee() through the superclass type confirmed dynamic dispatch was working as intended.

Learning Outcomes: This assignment deepened my understanding of when to use inheritance over composition, and gave me practical experience designing an abstract class whose only abstraction point is a single method, and keeping the hierarchy simple and easy to extend.

SDG Connection: The fee-waiver mechanism I implemented for SeniorPatient and EmergencyPatient directly supports SDG 10 (Reduced Inequalities) by encoding equitable pricing as a programmatic policy rather than inconsistent manual discretion.

CO Attainment: This work demonstrates CO1 — class design, inheritance, and polymorphism — assessed at Bloom's L3-L4.

Member 2 --- Kaviya Kumara | Roll No: 192425055

Design and Development Decisions: I was responsible for implementing the pharmacy inventory subsystem. Hashtable was deliberately chosen over a plain HashMap for medicine stock because Hashtable's full method-level synchronisation was sufficient for the concurrency level in this assignment and kept the implementation within the prescribed Collection Framework scope, and without needing an external lock.

Challenges Faced and Solutions: Triggering the low-stock alert at the right point required care — checking the reorder threshold before the stock update would use a stale value, while checking too late could miss the boundary case where dispensing brings the stock to exactly the reorder level. Moving the check to immediately after the Hashtable update fixed this.

Learning Outcomes: I learned that Hashtable's synchronised methods are a convenient way to achieve thread safety on a single shared map without hand-writing lock/unlock pairs, and I gained a clearer appreciation of why generics (Hashtable) eliminate unchecked-cast warnings and catch type errors at compile time rather than run time.

SDG Connection: The low-stock alert system I built directly supports SDG 3 by ensuring hospital pharmacies are restocked before critical medicines run out, and reduces dependency on manual stock checks, which also supports SDG 9's call for resilient infrastructure.

CO Attainment: My contribution addresses CO2 — Collection Framework usage with Hashtable and generics — assessed at Bloom's L3-L4.



Member 3 ---Sri vardhan | Roll No:192325006

Design and Development Decisions: I implemented the three user-defined exception classes and the appointment-booking and cancellation logic. Each exception extends RuntimeException rather than a checked Exception so that the menu-driven flow is not forced to declare throws clauses everywhere, while still requiring an explicit try-catch around the specific operations that can fail.

Challenges Faced and Solutions: The most significant challenge was making the waitlist-promotion logic race-condition-free. Early testing without synchronisation allowed two near-simultaneous booking calls to both read the same slot count, both decide a slot was free, and both attempt to book it. Wrapping the entire read-check-book sequence inside a synchronized block, and calling notifyAll() only after a successful booking, eliminated the race.

Learning Outcomes: I gained a concrete, tested understanding of why a check-then-act sequence on shared state must be atomic under concurrency, not just individually thread-safe — this is a distinction that is easy to state abstractly but only really sinks in after reproducing the bug and then fixing it.

SDG Connection: The correct, race-free waitlist-promotion logic I built ensures no patient is unfairly denied a slot due to a timing bug, which supports SDG 3's goal of equitable access to healthcare services.

CO Attainment: This work demonstrates CO3 — exception handling and synchronisation-safe service logic — assessed at Bloom's L3-L4.

Member 4 --- Venkata Anirudh | Roll No:192572110

Design and Development Decisions: I implemented the `NotificationDispatchThread` and designed the ten test cases in Section 7. I chose a `wait()/notify()` pattern over a polling loop specifically to avoid busy-waiting, because a naive `while(true)` polling loop would consume CPU time even when there was nothing to dispatch.

Challenges Faced and Solutions: Capturing deterministic, reproducible screenshots of a genuinely concurrent system was harder than expected, because thread interleaving can vary between runs. I addressed this by inserting short, deliberate delays around the operations being demonstrated so that the console output for each scenario in Section 8 appears in a stable, readable order while still being produced by real concurrent threads rather than a single-threaded simulation.

Learning Outcomes: I learned that `wait()` must always be called inside a while loop re-checking its condition, never inside a plain `if`, because a thread can wake spuriously or be notified when the condition it was waiting for is no longer true by the time it resumes — a subtlety that is easy to get wrong and hard to detect without deliberately stress-testing the concurrent paths.

SDG Connection: The automated, low-latency appointment-reminder notification I built aligns with SDG 3 by helping ensure patients do not miss consultations due to a lack of timely reminders.

CO Attainment: My contribution addresses CO3 — multithreading with synchronised methods and `wait()/notify()` inter-thread communication — assessed at Bloom's L3-L4.\